App Dev Project Report

1. Student Details

Name: Tushar Kant Pradhan

Roll Number: 23F2002972

Email: 23F2002972@ds.study.iitm.ac.in

About Me: I am a student at the IIT Madras BS Degree program with a deep interest in backend web development and architecture. I strive to engineer robust, role-based applications that solve complex management and operational challenges cleanly and securely.

2. Project Details

Project Title: TrekMap (Trekking Management Application)

Problem Statement:
To design and build a secure, web-based management application that seamlessly coordinates trekking operations, enabling administrators to oversee treks and staff, guides to manage field operations, and clients to discover and book adventures.

Approach:
The application was constructed using Flask as the backend framework with a strict Model-View-Controller (MVC) architecture. Blueprints were utilized to segregate routing logically among distinct user roles (Clients, Guides, Managers). The core logic executes entirely on the server side using Python and an SQLite database, ensuring a highly secure environment completely free of client-side JavaScript dependencies.

3. AI/LLM Declaration

It is necessary to state that I have used AI for just refinement and making the code look structured. All concepts and logic are my own. I have merely taken the help of AI to make the application structure efficient, and some direct UI touches have been done. There is 0.1% or NO usage of AI in the fundamental architecture and intelligent implementation of this project.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework
Flask-SQLAlchemy	Object Relational Mapper for SQLite database
Jinja2	Template engine for rendering dynamic HTML pages
Bootstrap 5 (Vapor)	Frontend styling and dark cyberpunk responsive design
Werkzeug	Secure password hashing (scrypt) and verification
SQLite	Lightweight local database for storing application data

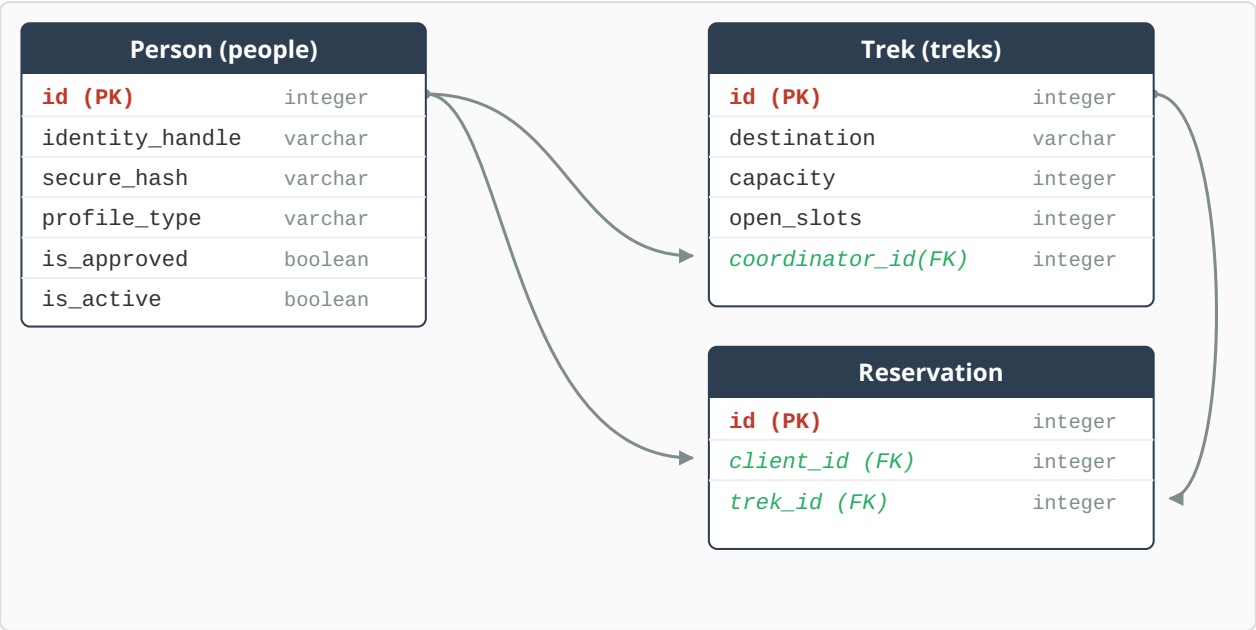
5. Database Schema / ER Diagram

Tables:

1. **Person (people)** - stores user and staff details (pid, identity_handle, secure_hash, profile_type, is_approved, is_active)
2. **Trek (treks)** - logs trek information (tid, destination, region, challenge_level, capacity, open_slots, dates, trek_status, coordinator_id)
3. **Reservation (reservations)** - records user bookings (rid, client_id, trek_id, timestamp)

Relationships:

- One-to-Many → Person (Guide) to Trek (coordinator_id)
- One-to-Many → Person (Client) to Reservation (client_id)
- One-to-Many → Trek to Reservation (trek_id)



6. API Resource Endpoints

Endpoint (Route)	Method	Description
/client/login	GET, POST	Authenticate user and generate session routing to role dashboard
/client/dash	GET	Fetch open treks and personal reservation history for user
/client/book/<tid>	POST	Add new reservation for a trek
/guide/dash	GET	Fetch assigned treks for the authenticated guide
/guide/update_trek/<tid>	POST	Update trek status and modify open slots
/admin/dash	GET	Superuser console fetching all treks, guides, and users
/admin/add_trek	POST	Create a new trek entity in the database

7. Architecture and Folder Structure

Architecture Overview:

```
TrekMap/
│
├── main.py                    ← Main entry point. Boots the Flask app.
│
├── Database/
│   └── models.py              ← SQLAlchemy models (Person, Trek, Reservation).
│
├── routes/
│   ├── auth_routes.py
│   ├── client_routes.py       ← Blueprint for client login, dashboard, booking
│   ├── guide_routes.py        ← Blueprint for guide dashboard, trek updates
│   └── admin_routes.py        ← Blueprint for manager dashboard, create treks
│
├── views/
│   ├── register.html          ← All Jinja2 HTML templates
│   ├── layout.html            ← Base template (navbar, Bootstrap Vapor CDN)
│   ├── auth.html              ← Login form page
│   ├── manager_dash.html      ← Manager dashboard
│   ├── guide_dash.html        ← Guide dashboard
│   └── client_dash.html       ← Client dashboard
│
├── style/                     ← Static files folder
│   └── brutal.css             ← Extra styling overrides
│
├── TMAS_sqlite.db             ← SQLite database file (auto-created)
└── Project Report.pdf         ← Complete guide & documentation
```

Implemented Features:

- Programmatic database auto-generation on initial boot
- Role-based access control (Manager, Guide, Client)
- Zero JavaScript frontend architecture for maximum client-side security
- Administrative master controls (Ban users, Approve guides)
- Dynamic trek capacity tracking and booking system
- Secure Scrypt password hashing and user session handling

8. Video Presentation

Drive Link:
https://drive.google.com/file/d/1IHgHLYvMD7raynJ7b-ZIou_1C5qc6VeX/view?usp=sharing